

Enunciado Evaluacion Parcial 1

Programacion Orientada a Objetos

FORMA A

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1102	Programacion Orientada a Objetos	EA1: Fundamentos de Programacion Orientada a Objetos
Tiempo	Modalidad de Trabajo	Indicadores de logro
3 bloques	Individual	IL 1.1 al IL 1.6

Condiciones de la evaluacion

Del entorno y la entrega

- La evaluacion es de caracter **individual**. No esta permitida ninguna forma de colaboracion entre estudiantes.
- No se permite el acceso a internet ni el uso de telefonos, relojes inteligentes ni ningun otro dispositivo externo.
- El programa debe compilar y ejecutarse correctamente. Una solucion que no compile no permite evidenciar ningun indicador de logro.
- El proyecto debe entregarse comprimido en formato ZIP, con tu nombre completo como nombre del archivo, en la actividad habilitada en AVA antes de que finalice la sesion.
- No se aceptan entregas por ningun otro medio ni fuera del horario establecido. La ausencia de entrega en AVA genera automaticamente la nota minima.

De tu solucion

- El codigo debe organizarse en clases separadas, respetando las convenciones de nomenclatura de Java: `PascalCase` para nombres de clase, `camelCase` para metodos y atributos.

- Puedes crear metodos o clases auxiliares siempre que no contradigan los requerimientos del caso.
 - No se admite el uso de herramientas de inteligencia artificial ni de recursos digitales externos.
 - Cualquier indicio de colaboracion no autorizada, uso de IA o plagio implica la anulacion inmediata de la evaluacion con nota minima de 1.0.
-

Instrucciones de entrega con Git

Antes de comenzar a codificar, debes configurar Git y preparar tu repositorio. Sigue estos pasos en orden:

Paso 1 — Configurar tu identidad en Git (si no lo has hecho previamente)

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu.correo@duocuc.cl"
```

Paso 2 — Clonar el repositorio de tu seccion

Seccion 004D:

```
git clone https://github.com/hachimaki-dev/004D_Desarrollo_orientado_a_Objeto_2026
```

Seccion 002D:

```
git clone https://github.com/hachimaki-dev/002D_desarrollo_orientado_a_objeto_2026
```

Paso 3 — Ingresar al directorio del repositorio clonado

```
cd 004D_Desarrollo_orientado_a_Objeto_2026
```

Ajusta el nombre del directorio segun tu seccion.

Paso 4 — Crear una rama nueva para tu entrega

```
git checkout -b evaluacion1/Nombre_Apellido
```

Ejemplo: `git checkout -b evaluacion1/Carlos_Orellana`

Paso 5 — Desarrollar tu solucion dentro de esta rama

Crea tus archivos `.java` y trabaja tu solucion normalmente.

Paso 6 — Guardar tus cambios en un commit

```
git add .  
git commit -m "Entrega EP1 - Nombre Apellido - Confirmo que esta es mi entrega f
```

Paso 7 — Publicar tu rama en el repositorio remoto

```
git push origin evaluacion1/Nombre_Apellido
```

Importante: Además de publicar tu rama en Git, debes comprimir tu proyecto en formato ZIP (con tu nombre completo como nombre del archivo) y subirlo al AVA antes de que finalice la sesión. Ambas entregas son obligatorias.

Caso de negocio

Una clínica veterinaria necesita un sistema de consola en Java para gestionar el registro de mascotas y sus consultas médicas. La clínica atiende dos tipos de mascotas: **mascotas domesticas** (perros, gatos) y **mascotas exóticas** (reptiles, aves, entre otros). Ambas comparten datos generales como nombre y especie, pero se diferencian en sus atributos específicos y en la forma de calcular el costo de una consulta.

El sistema debe permitir registrar mascotas, listar las mascotas registradas, buscar mascotas por nombre, registrar consultas médicas y ver un resumen de los pacientes. Todas las entradas del usuario deben estar protegidas contra errores de ingreso.

Estructura de clases

El siguiente diagrama muestra la jerarquía de clases a implementar. La clase `Mascota` es **abstracta** y define la estructura base. Las clases `MascotaDomestica` y `MascotaExotica` heredan de ella y sobrescriben el cálculo del costo de consulta.

Mascota (abstracta)

- nombre: String
- especie: String
- consultasRealizadas: int

- + Mascota(nombre, especie)
- + getNombre(): String
- + getEspecie(): String
- + getConsultasRealizadas(): int
- + setConsultasRealizadas(int): void
- + *calcularCostoConsulta(): int*
- + mostrarInfo(): String

▲ extends

MascotaDomestica

- vacunasAlDia: boolean

- + MascotaDomestica(nombre, especie, vacunasAlDia)
- + getVacunasAlDia(): boolean
- + calcularCostoConsulta(): int
- + mostrarInfo(): String

MascotaExotica

- costoPermiso: int

- + MascotaExotica(nombre, especie, costoPermiso)
- + getCostoPermiso(): int
- + calcularCostoConsulta(): int
- + mostrarInfo(): String

Nota: Los costos se manejan como `int` (pesos chilenos enteros, sin decimales). El atributo `consultasRealizadas` se inicializa en **0** dentro del constructor y no se recibe como parametro. Se incrementa cada vez que se registra una consulta para esa mascota.

Requerimientos funcionales

A continuacion se detalla cada requerimiento. Las etiquetas **IL** indican que indicador de logro se evalua en cada parte.

R1 — Comentarios de paradigma **IL 1.1**

Al inicio de la clase `Main.java`, incluye un bloque de comentarios que explique:

- Que paradigma de programacion se esta usando y sus características principales.
- Al menos **2 diferencias concretas** entre el paradigma orientado a objetos (Java) y el paradigma que usabas antes (Python funcional). Por ejemplo: tipado, estructura del programa, compilacion vs. interpretacion, o forma de organizar el codigo.

R2 — Clase abstracta Mascota **IL 1.3** **IL 1.4**

Implementa la clase abstracta `Mascota` segun el diagrama UML:

- Atributos **privados**: `nombre` (String), `especie` (String), `consultasRealizadas` (int).
- Constructor que reciba `nombre` y `especie`. El atributo `consultasRealizadas` se inicializa en 0.
- Getters para los tres atributos y setter para `consultasRealizadas`.
- Metodo abstracto `calcularCostoConsulta()` que retorna `int`.
- Metodo `mostrarInfo()` que retorna un `String` con el nombre, especie, costo de consulta y cantidad de consultas realizadas. Este metodo puede ser sobrescrito por las clases hijas para agregar informacion adicional.

R3 — Clase MascotaDomestica IL 1.3 IL 1.4

Hereda de `Mascota` . Representa una mascota domestica (perro, gato, etc.).

- Atributo privado adicional: `vacunasAlDia` (boolean).
- Constructor que reciba los 3 parametros (2 del padre + `vacunasAlDia`). Usa `super()` .
- Getter para `vacunasAlDia` .
- `calcularCostoConsulta()` : si `vacunasAlDia` es `true` , retorna **15000**. Si es `false` , retorna **20000** (recargo por actualizacion de vacunas).
- `mostrarInfo()` : sobrescribe (`@Override`) el metodo del padre. Agrega el tipo (`"Domestica"`) y el estado de vacunas al texto retornado.

R4 — Clase MascotaExotica IL 1.3 IL 1.4

Hereda de `Mascota` . Representa una mascota exotica (reptil, ave, etc.).

- Atributo privado adicional: `costoPermiso` (int, costo del permiso SAG requerido para la atencion).
- Constructor que reciba los 3 parametros (2 del padre + `costoPermiso`). Usa `super()` .
- Getter para `costoPermiso` .
- `calcularCostoConsulta()` : retorna `15000 + costoPermiso` .
- `mostrarInfo()` : sobrescribe (`@Override`) el metodo del padre. Agrega el tipo (`"Exotica"`) y el costo del permiso al texto retornado.

R5 — Menu de consola con estructuras de control IL 1.2

La clase `Main` debe presentar un menu de consola que se repita hasta que el usuario elija salir. Usa un bucle `while` y `switch` para controlar las opciones:

```
≡≡≡ CLINICA VETERINARIA ≡≡≡
1. Registrar mascota
2. Listar mascotas
3. Buscar mascota por nombre
4. Registrar consulta
5. Resumen de pacientes
6. Salir
Seleccione una opcion: _
```

Las opciones invalidas deben mostrar un mensaje y volver al menu sin que el programa se detenga.

R6 — Registrar mascota (con submenu) IL 1.2 IL 1.6

Al seleccionar "Registrar mascota", el sistema muestra un submenu para elegir entre Domestica y Exotica. Luego solicita los datos correspondientes y agrega la mascota a una coleccion `ArrayList<Mascota>`.

- Para mascotas domesticas, se solicita si las vacunas estan al dia. El usuario ingresa `"si"` o `"no"` y se almacena como `boolean`.
- Para mascotas exoticas, se solicita el costo del permiso SAG como un valor entero.

R7 — Listar mascotas (polimorfismo en accion) IL 1.5 IL 1.6

Recorre la coleccion `ArrayList<Mascota>` e imprime la informacion de cada mascota usando `mostrarInfo()`.

Cada mascota se muestra con su formato propio (domestica o exotica) sin necesidad de hacer `instanceof`. Esto es polimorfismo: el mismo metodo produce resultados distintos segun el tipo real del objeto.

Si la lista esta vacia, mostrar un mensaje indicando que no hay mascotas registradas.

R8 — Buscar mascota por nombre

IL 1.2

IL 1.6

El usuario ingresa un texto y el sistema busca mascotas cuyo nombre contenga ese texto (busqueda parcial, sin distinguir mayusculas/minusculas). Muestra todos los resultados encontrados o un mensaje si no hay coincidencias.

R9 — Registrar consulta

IL 1.2

IL 1.5

IL 1.6

El sistema lista las mascotas con su indice. El usuario selecciona una mascota e ingresa una descripcion del procedimiento realizado. El sistema valida:

- Que el numero de mascota sea valido (exista en la lista).
- Que la descripcion no este vacia.

Si la consulta es valida, incrementa el contador `consultasRealizadas` de la mascota y muestra un recibo con el nombre de la mascota, el procedimiento descrito, el costo de la consulta (calculado con `calcularCostoConsulta()`) y el total de consultas acumuladas.

R10 — Resumen de pacientes

IL 1.2

IL 1.5

IL 1.6

Muestra un resumen que incluya:

- Total de mascotas registradas.
- Cantidad de mascotas domesticas y cantidad de mascotas exoticas (usa `instanceof`).
- Total de consultas realizadas (suma de `consultasRealizadas` de todas las mascotas).
- Ingresos totales: la suma de `calcularCostoConsulta()` * `consultasRealizadas` de cada mascota.

R11 — Manejo de excepciones IL 1.5

Toda entrada numerica del usuario debe estar protegida con `try-catch` . El programa **nunca debe detenerse** por un error de ingreso. Situaciones que deben manejarse:

- El usuario escribe texto cuando se espera un numero (costo de permiso, indice de mascota).
- El usuario ingresa un indice fuera de rango al registrar una consulta.
- El usuario ingresa una opcion de menu que no existe.

En cada caso, muestra un mensaje claro al usuario y vuelve al menu sin interrumpir la ejecucion.

Ejemplo de ejecucion esperada

Este ejemplo muestra como deberia verse la interaccion con el programa. Tu implementacion debe producir una salida similar (no necesariamente identica en formato).

≡ CLINICA VETERINARIA ≡

1. Registrar mascota
2. Listar mascotas
3. Buscar mascota por nombre
4. Registrar consulta
5. Resumen de pacientes
6. Salir

Seleccione una opcion: 1

--- Tipo de mascota ---

1. Domestica
2. Exotica

Seleccione: 1

Nombre: Luna

Especie: Gato

Vacunas al dia (si/no): si

[OK] Mascota domestica registrada.

Seleccione una opcion: 1

--- Tipo de mascota ---

1. Domestica
2. Exotica

Seleccione: 2

Nombre: Kiwi

Especie: Iguana

Costo del permiso SAG (\$): 8000

[OK] Mascota exotica registrada.

Seleccione una opcion: 2

--- MASCOTAS REGISTRADAS ---

[1] Domestica | Luna | Especie: Gato | Vacunas: Si | Costo consulta: \$15000 | Costo SAG: \$0
[2] Exotica | Kiwi | Especie: Iguana | Permiso: \$8000 | Costo consulta: \$23000 | Costo SAG: \$8000

Seleccione una opcion: 4

--- REGISTRAR CONSULTA ---

[1] Luna (Gato) | Consultas: 0

[2] Kiwi (Iguana) | Consultas: 0

Ingrese numero de mascota: 1

Descripcion del procedimiento: Control general y desparasitacion

Consulta registrada.

Mascota: Luna

Procedimiento: Control general y desparasitacion

Costo de la consulta: \$15000

Consultas acumuladas: 1

Seleccione una opcion: 5

--- RESUMEN DE PACIENTES ---

Total de mascotas: 2

Mascotas domesticas: 1

```
Mascotas exoticas: 1
Total de consultas realizadas: 1
Ingresos totales: $15000

Seleccione una opcion: 3

--- BUSCAR MASCOTA ---
Ingrese texto a buscar: lu
Resultados:
    Domestica | Luna | Especie: Gato | Vacunas: Si | Costo consulta: $15000 | Cons

Seleccione una opcion: 4
Ingrese numero de mascota: abc
Error: Debe ingresar un numero valido.

Seleccione una opcion: 6
Programa finalizado.
```

Mapa de indicadores de logro

Esta tabla resume donde se evalua cada indicador. Usala como checklist antes de entregar.

IL	Indicador	Donde se evidencia
IL 1.1	Identificar el paradigma OO y diferenciarlo de otros	Comentarios en <code>Main.java</code> (R1)
IL 1.2	Estructuras de control y metodos con parametros y retorno	Menu while/switch (R5), submenu (R6), busqueda (R8), registro de consulta (R9)
IL 1.3	Clases con encapsulamiento y colaboracion	Atributos privados, getters/setters, constructores en <code>Mascota</code> , <code>MascotaDomestica</code> , <code>MascotaExotica</code> (R2, R3, R4)
IL 1.4	Herencia, clases abstractas, jerarquias de tipos	Clase abstracta <code>Mascota</code> , herencia con <code>extends</code> , <code>super()</code> , metodo abstracto (R2, R3, R4)
IL 1.5	Polimorfismo y manejo de excepciones	Recorrido polimorfico del ArrayList (R7), <code>@Override</code> de metodos (R3, R4), try-catch (R11), instanceof en resumen (R10)
IL 1.6	Colecciones para almacenar y gestionar datos	<code>ArrayList<Mascota></code> como registro, agregar, iterar, buscar, modificar consultas (R6, R7, R8, R9, R10)

Antes de entregar

Debe funcionar

El programa compila con `javac *.java` y se ejecuta con `java Main` sin errores.

Cada opcion del menu hace lo que se pide.

Los metodos sobrescritos (`@Override`) funcionan correctamente en cada subclase.

No debe romperse

Escribir "abc" cuando se pide un numero.

Seleccionar una mascota que no existe al registrar consulta.

Buscar una mascota cuando la lista esta vacia.

Seleccionar una opcion de menu inexistente.

Archivos a entregar

Comprime en formato **ZIP** todos los archivos `.java` de tu proyecto. Como minimo debes tener:

- `Main.java`
- `Mascota.java`
- `MascotaDomestica.java`
- `MascotaExotica.java`

Importante: No se permite el uso de herramientas de IA, acceso a internet, ni dispositivos externos durante la evaluacion. Cualquier indicio de colaboracion no autorizada o uso de IA implica la anulacion de la evaluacion con nota minima 1.0.

Lee bien cada requerimiento, planifica antes de codificar y prueba tu programa antes de entregar.

